

Engram 条件记忆模块详解报告

引言

在大型语言模型（LLM）领域，**稀疏性（sparsity）**是提升模型容量的重要策略。经典的混合专家（Mixture-of-Experts, MoE）通过“条件计算”在每个输入位置只激活少数专家，从而在不增加每个 token FLOPs 的情况下扩展模型容量。但单靠 MoE，模型仍需在内部用深层计算重建事实知识和局部模板，这既浪费计算，也占用了注意力容量。DeepSeek 的论文《Conditional Memory via Scalable Lookup》提出了一种新的稀疏轴 **条件记忆（conditional memory）**，并将其具体化为 **Engram 模块**，以结构化查表的方式存储和检索局部模式。当 Engram 与 MoE 协同使用时，可在固定 FLOPs 下优化参数分配，从而显著提升模型在知识、推理、代码和数学等任务的表现 ^①。

模块结构概览

Engram 模块仅插入部分 Transformer 层（例如第 2 层和第 15 层），通过结构化查表而非深层计算来处理局部依赖，其计算分为**检索（Retrieval）**和**融合（Fusion）**两个阶段 ^②。

1. 检索阶段：从后缀 N-gram 查表

- **词表压缩**：标准 subword tokenizer 会为同义异形词（如 “Apple” 与 “apple”）分配不同 ID，导致稀疏表容量浪费。Engram 在预处理阶段构建一个词表投影函数 $P: V \rightarrow V'$ ，基于 NFKC 归一化和小写化将原始 token ID 映射到 **规范化 ID**，从而将有效词表规模减少约 23% ^③。
- **后缀 N-gram 构建**：对于序列中第 t 个位置，取由规范化 token 组成的后缀 N-gram $g_{t,n} = (w'_{t-n+1}, \dots, w'_t)$ （论文默认使用 2-gram 和 3-gram）。
- **多头哈希（Multi-Head Hashing）**：直接为所有可能的 N-gram 建表不可行，Engram 使用多个哈希头将每个 N-gram 映射到不同的素数大小的哈希表。每个哈希头 (n, k) 使用乘法-异或（multiplicative-XOR）函数 $\phi_{n,k}(g_{t,n})$ 产生索引 $z_{t,n,k}$ ，然后从嵌入表 $E_{n,k}$ 取出向量 $e_{t,n,k}$ 。最终记忆向量 e_t 是所有 n 和 k 的嵌入拼接 ^④。

2. 融合阶段：与隐藏状态的上下文融合

- **键值投影与门控**：取当前层的隐藏状态 h_t 作为 Query，对记忆向量 e_t 通过线性映射分别生成 Key 和 Value，然后对二者施加 RMSNorm 进行归一化。通过计算两者的点积并除以 $\sqrt{d}$ （其中 d 为隐藏维度）再经过 Sigmoid 得到一个标量门控 α_t 。该门控反映了检索记忆与当前语境的匹配度，决定记忆向量贡献的大小 ^⑤。
- **深度可分离因果卷积**：门控后的 Value 向量先通过 SiLU 非线性，再经过一个卷积核宽度小、dilation 等于最大 N-gram 阶数的一维深度可分离因果卷积，再加入残差。这一步既混合了相邻位置的查表结果，又保持计算开销极小 ^⑥。

- **多分支集成**：论文的默认骨干使用 *Manifold-Constrained Hyper-Connections* (mHC) 多分支结构。为了兼顾表达力和效率，Engram 在多分支框架下采用 **共享静态嵌入与 Value 投影、每个分支独立的 Key 投影**。这种设计允许在 FP8 矩阵乘法中融合多个分支的投影，充分利用 GPU 计算资源 ⁷。

3. 系统实现与部署优势

Engram 检索索引完全由输入 token 决定，不依赖动态隐藏状态，这使得 **检索可预取**：推理阶段可以在计算前预先从 CPU 内存或高速缓存加载相关嵌入，并与前层计算重叠，从而大幅降低延迟 ⁸。论文还指出：

- **训练阶段**：将大表分片至多张 GPU 并使用 All-to-All 通信收集激活行，Embedding 总容量随 GPU 数线性扩展 ⁹。
- **推理阶段**：将 Engram 表离线到主机内存，通过 PCIe 异步预取嵌入，并利用前层计算掩蔽通信延迟；实验证明即便 100B 参数表完全在主机内存，推理吞吐率损失也不到 3% ¹⁰ ¹¹。
- **多级缓存**：自然语言 N-gram 的访问呈 Zipf 分布，因此可将访问频繁的嵌入缓存于 GPU HBM 或 host DRAM，将长尾存储在 NVMe SSD，从而在保持延迟的前提下进一步扩展容量 ¹²。

稀疏性分配与缩放定律

1. 稀疏性分配问题

论文将总参数分为 **激活参数 (act)** 与 **非激活参数 (sparse)**。MoE 的非激活参数来自未被路由的专家；Engram 的非激活参数来自未检索的嵌入。作者设定一个 **分配比** $\rho \in [0, 1]$ ，表示将非激活预算分配给 MoE 的比例：当 $\rho = 1$ 时为纯 MoE，当 $\rho = 0$ 时为纯 Engram ¹³。在固定总参数和 FLOPs (iso-参数、iso-FLOPs) 下，通过调整 MoE 专家数量与 Engram 表容量进行实验：

- **U 形关系**：图 3 左部显示，验证损失随着 ρ 呈明显的 U 形曲线；当只给 MoE 分配时 ($\rho = 100$) 效果次优，当分配 20-25% 非激活参数给 Engram ($\rho \approx 75$) 时验证损失最小 ¹⁴。
- **解释**：纯 MoE 模型无法存储静态模式，迫使网络在浅层通过计算重建，浪费深度；而纯 Engram 则缺乏条件计算能力，导致动态推理受损。最佳分配反映了 Engram 与 MoE 的互补性 ¹⁵。

2. 无限记忆尺度

在固定 MoE backbone 的情况下，作者将 Engram 表容量从 2.58×10^5 扩展到 1.0×10^7 个槽（总计约 13B 参数）。结果表明：**验证损失随表容量呈对数线性下降**，即 Engram 的收益与表大小对数成线性关系；相比简单平均 N-gram 嵌入 (OverEncoding)，Engram 能在相同内存预算下实现更大的规模潜力 ¹⁶。这说明 Engram 提供了一个可持续扩展的参数旋钮，几乎不增加计算开销。

大规模预训练与实验结果

1. 模型配置

作者使用 DeepSeek-v3 的 128k 词表，在 30 层 Transformer 结构上，训练了四种模型：

1. **Dense-4B**：作为密集基线，包含 4.1B 总参数，激活参数约 3.8B。

2. **MoE-27B**：在每层 FFN 处替换为 MoE 模块，包含 26.7B 总参数，其中专家个数为 72（top-6 路由）¹⁷。
3. **Engram-27B**：将 MoE-27B 的部分专家（72→55）释放出的参数（5.7B）用于 Engram 表，并在第 2、15 层插入 Engram 模块¹⁸。
4. **Engram-40B**：与 Engram-27B 的 backbone 相同，但将 Engram 表扩大至 18.5B 参数，模型总参数 39.5B¹⁹。

所有模型激活参数均为 3.8B，训练 tokens 为 262B，相同优化超参。

2. 性能对比 (Table 1)

在语言建模、知识推理、阅读理解、代码与数学等任务上，Engram 模型表现出显著优势（下列指标均为 Engram-27B 与 MoE-27B 的差异）：

- **语言建模**：在 Pile 验证集上，Engram-27B 的损失 1.622 显著优于 MoE-27B 的 1.634，Engram-40B 进一步降至 1.610²⁰。
- **知识与推理**：MMLU、CMMLU、MMLU-Pro 等考试型基准均有大幅提升。例如 MMLU 从 57.4→60.4 (+3.0)，CMMLU 从 57.9→61.9 (+4.0)，AGIEval 从 38.6→41.8 (+3.2)²¹。
- **阅读理解**：在 DROP、RACE Middle/High 等阅读任务上，Engram-27B 获得 3–4 点的 F1 或准确率提升²²。
- **代码与数学**：HumanEval Pass@1 从 37.8→40.8 (+3.0)，MBPP 从 46.6→48.2 (+1.6)，GSM8K 从 58.4→60.6 (+2.2)，MATH 从 28.3→30.7 (+2.4)²³。

此外，作者指出这些提升并非仅来自知识检索；Engram 在一般推理和代码/数学任务上的增益更为明显，这表明它通过解放网络的早期层和注意力容量，为复杂推理腾出了空间²⁴。

3. 长上下文训练 (Long-Context Training)

为了测试 Engram 在长序列情境下的优势，作者在预训练后利用 YaRN 方法扩展上下文窗口至 32 k tokens，并在长文本（书籍、论文、代码、链式思考）和 Needle-in-a-Haystack 检索任务上评估模型。

结果显示，在相同预训练损失下（46k 步），Engram-27B 在长文本困惑度和 RULER 子任务上全面优于 MoE-27B。例如，在多查询 NIAH 子任务中准确率从 84.2% 提升至 97.0%，变量跟踪提升至 87.2%²⁵。作者认为这是因为 Engram 把局部依赖托管给查表，释放了注意力容量，使模型能更好地聚焦全局上下文。

机制分析与消融实验

1. 深度提升的机制

- **LogitLens 分析**：通过投影每层隐状态到最终输出分布计算 KL 散度，发现 Engram 模型在早期层的 KL 散度显著更低，说明模型更早收敛到正确预测。这表明 Engram 可以代替几层计算来快速重建静态知识²⁶²⁷。
- **CKA 表征对齐**：利用 Centered Kernel Alignment 比较 Engram 与 MoE 的层表示，观察到 Engram 的浅层表示与 MoE 的深层表示对齐曲线向上移。例如 Engram-27B 第 5 层的表示与 MoE-27B 第 12 层相似，这意味着 Engram 有效提升了网络深度²⁸。

这些发现支持论文的核心观点：通过显式的知识查表，模型无需在浅层通过 Attention/FFN 逐渐组合局部模式，从而将“有效深度”腾给复杂推理。

2. 结构消融与插入位置

作者对一个 12 层、3 B MoE 模型引入 1.6 B 参数的 Engram，并探索插入层的位置及各组件作用：

- **插入层扫描**：将所有内存集中于单个 Engram 模块并从第 1 层移至第 12 层，发现第 2 层是最佳插入点（Val Loss = 1.770），说明经过一次注意力汇聚后即可获得足够的语境做门控；更深的层由于缺乏局部模式的重建需求，效果反而下降²⁹。
- **分层插入**：将同样的内存预算拆分为两个 Engram 模块分别置于第 2 层和第 6 层，性能进一步提升（Val Loss = 1.768），兼顾了早期卸载局部模式和后期更精细的门控³⁰。
- **组件消融**：去除多分支融合、上下文门控或词表压缩都会导致验证损失明显回退，说明这些设计是 Engram 的核心贡献；而移除深度卷积或添加 4-gram 等改动影响较小³¹。

3. 功能敏感性

在推理阶段关闭 Engram 输出并保持 backbone 不变，分析各任务对该模块的依赖。结果显示：

- **事实知识任务**（如 TriviaQA、PopQA 等）性能仅保留 29–44%，表明 Engram 是主要的知识存储库；
- **阅读理解和常识推理任务**保留 81–93%，说明这些任务主要依赖 backbone 的注意力机制³²。

这一对比强化了 Engram 专注静态知识而非动态推理的角色定位。

4. 系统效率分析

在一个原型系统 nano-vLLM 上，将一个 100 B 参数的 Engram 层完全放在主机内存，并插入到 Transformer 的第二层。结果显示相较于 Dense-4B 和 Dense-8B 基线模型，推理吞吐率仅下降约 2.8%，证明通过预取和通信-计算重叠可以消除几乎所有的内存访问开销¹¹。

GitHub 仓库与示例

DeepSeek-AI 在 GitHub 发布了 Engram 的官方实现³³。仓库包括：

- **论文 PDF 和绘图**，用于理解架构与实验结果；
- `engram_demo_v1.py`，示例脚本演示检索和融合的数据流，使用模拟的 Attention/MoE/mHC 模块；
- **Apache-2.0 许可**，允许研究人员自由使用；
- **快速开始指南**，建议使用 Python 3.8+ 和 PyTorch，并提供安装指令及示例运行命令³⁴。

README 概述了论文的关键贡献：提出稀疏性分配的 U 形定律、在 iso-参数和 iso-FLOPs 条件下验证 Engram 的有效性、通过机制分析解释其作用，并强调系统效率优化³⁵。

结论与展望

Engram 模块通过“存取分离”的设计，在不显著增加计算的情况下引入了条件记忆这一稀疏维度，为 LLM 提供了重要补充：

- 1. **结构优势**：通过 tokenizer 压缩和多头哈希实现 $O(1)$ 检索，将局部、静态模式存储在大型嵌入表中，并通过上下文文化门控及轻量卷积有效融合。其多分支集成与可预取特性使它在系统层面极为友好，能够利用多级缓存和异步传输扩展至百亿参数表¹⁰。
- 2. **性能提升**：在固定 FLOPs 和激活参数条件下，Engram-27B 在知识、推理、代码、数学任务上全面超越同规模 MoE；稀疏性分配实验表明最佳性能在约 20–25% Engram、75–80% MoE 的组合¹⁴。
- 3. **机制解释**：LogitLens 和 CKA 分析表明 Engram 通过查表加速浅层特征收敛，相当于“提高网络有效深度”，并释放注意力容量用于全局上下文，从而改善长上下文任务²⁶²⁸。
- 4. **系统可扩展性**：由于检索地址确定，Engram 可脱离 GPU HBM 而存储在 host 内存或 NVMe SSD，支持软硬件协同优化；实验证明 100 B 参数表在推理中几乎无吞吐损失¹⁰¹¹。

未来方向：

- **更高阶 N-gram 和适应其他模态**：论文中发现 4-gram 在固定预算下略有下降，但在更大表规模下或许会发挥作用；此外，可以探索 Engram 在多模态模型或语音模型中的应用。
- **动态词表压缩与语义聚类**：当前压缩依赖预定义规则，未来可使用可学习的投影或语义聚类进一步提升词表密度。
- **跨域迁移**：如通信信道处理等领域的数据具有连续特征，如何定义合适的离散化与局部模式，使 Engram 思想迁移至非语言任务，是一个有趣的研究方向。

总体而言，Engram 在理论、工程与应用层面都展现出巨大潜力，它与 MoE 互补并提供了一条新的稀疏扩展道路，有望成为下一代大模型的重要组成模块。

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30
31 32 Conditional Memory via Scalable Lookup: A New Axis of Sparsity for Large Language Models
<https://arxiv.org/pdf/2601.07372>
33 34 35 GitHub - deepseek-ai/Engram: Conditional Memory via Scalable Lookup: A New Axis of Sparsity for Large Language Models
<https://github.com/deepseek-ai/Engram>